CS 521: Fall 2026
Homework 2

Atharva Bhatt (bhatt13@illinois.edu)

Problem 1. (Operations on Complex Intervals)

The problem defines a complex interval as a rectangle, $I = [a, b] + i[c, d]$, so the result is computed by box propagation.

(a) Addition

Real and imaginary parts add independently, and every value in each sum interval is attained, so

$$I_1 + I_2 = [a_1 + a_2,\ b_1 + b_2] + i\,[c_1 + c_2,\ d_1 + d_2].$$

This is exact.

(b) Multiplication

Take $z_1 = x_1 + iy_1$ and $z_2 = x_2 + iy_2$ with $x_1 \in [a_1, b_1]$, $y_1 \in [c_1, d_1]$, $x_2 \in [a_2, b_2]$, $y_2 \in [c_2, d_2]$. Then

$$z_1 z_2 = (x_1 x_2 - y_1 y_2) + i\,(x_1 y_2 + y_1 x_2).$$

We have disjoint variables within each component. In the real part, $x_1 x_2$ uses $\{x_1, x_2\}$ and $y_1 y_2$ uses $\{y_1, y_2\}$. In the imaginary part, $x_1 y_2$ uses $\{x_1, y_2\}$ and $y_1 x_2$ uses $\{y_1, x_2\}$. So the two terms within each component can be chosen independently.

Now we define the endpoint products:

$$\begin{aligned}
P &= \{a_1a_2,\ a_1b_2,\ b_1a_2,\ b_1b_2\}, & x_1x_2 &\in [\min P,\ \max P],\\
Q &= \{c_1c_2,\ c_1d_2,\ d_1c_2,\ d_1d_2\}, & y_1y_2 &\in [\min Q,\ \max Q],\\
R &= \{a_1c_2,\ a_1d_2,\ b_1c_2,\ b_1d_2\}, & x_1y_2 &\in [\min R,\ \max R],\\
S &= \{c_1a_2,\ c_1b_2,\ d_1a_2,\ d_1b_2\}, & y_1x_2 &\in [\min S,\ \max S].
\end{aligned}$$

Since the two terms in both the Real and Imaginary component are independent, their extremes can occur simultaneously. For the real part, the lower bound is $\min P - \max Q$, i.e., we can choose the x endpoints giving $\min P$ and, independently, the y endpoints giving $\max Q$. The upper bound is $\max P - \min Q$. Similarly, the imaginary part has lower bound $\min R + \min S$ and upper bound $\max R + \max S$.

The box containing the complex product is

$$I_1 \cdot I_2 = [\min P - \max Q,\ \max P - \min Q] + i\,[\min R + \min S,\ \max R + \max S].$$

The interval is precise, since all four endpoints are attained by the product in some case.

Problem 2. (Certification using Zonotopes)

(a) The 2D shape of ϕ

The zonotope is

$$\hat{x}_1 = 2\varepsilon_1 + \varepsilon_2 + 4, \tag{1}$$
$$\hat{x}_2 = \varepsilon_1 + 2\varepsilon_2 + 3, \qquad \varepsilon_1, \varepsilon_2 \in [-1, 1]. \tag{2}$$

Eliminating one noise symbol at a time gives

$$2\hat{x}_1 - \hat{x}_2 = 3\varepsilon_1 + 5, \tag{3}$$
$$2\hat{x}_2 - \hat{x}_1 = 3\varepsilon_2 + 2. \tag{4}$$

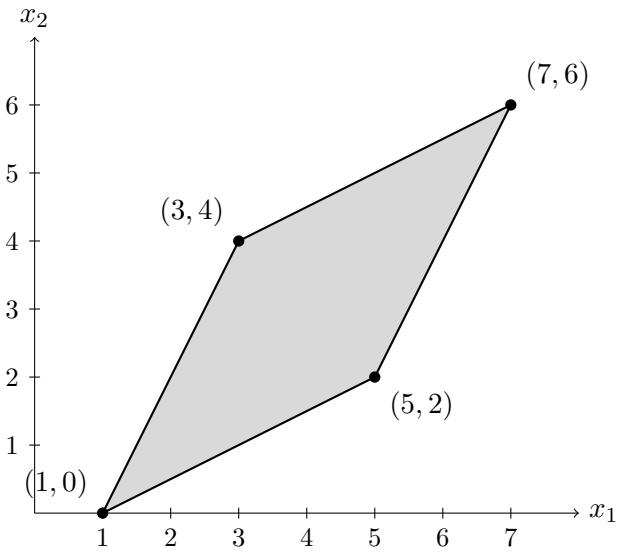
Since $\varepsilon_1, \varepsilon_2 \in [-1, 1]$, the zonotope is the region

$$2 \le 2\hat{x}_1 - \hat{x}_2 \le 8,$$
$$-1 \le 2\hat{x}_2 - \hat{x}_1 \le 5.$$

Evaluating at $(\varepsilon_1, \varepsilon_2) \in \{(-1,-1),\ (1,1),\ (1,-1),\ (-1,1)\}$:

$$(-1,-1) \mapsto (1,0), \quad (1,1) \mapsto (7,6), \quad (1,-1) \mapsto (5,2), \quad (-1,1) \mapsto (3,4).$$

Hence ϕ is a parallelogram with vertices $(1,0)$, $(5,2)$, $(7,6)$ and $(3,4)$:



(b) Zonotope transformers for the affine and ReLU layers

Yes, it is possible to prove the property. The affine layer is handled exactly, the ReLU on x_3 is exact, and the ReLU on x_4 is handled with Zonotope ReLU transformer. The details are given below.

Affine layer. Here $x_3 = x_1 + 2x_2$ and $x_4 = x_2 - x_1$. Using the zonotope (1), (2):

$$x_3 = x_1 + 2x_2 = 4\varepsilon_1 + 5\varepsilon_2 + 10, \tag{5}$$
$$x_4 = x_2 - x_1 = \varepsilon_2 - \varepsilon_1 - 1. \tag{6}$$

Since $\varepsilon_1, \varepsilon_2 \in [-1, 1]$, the bounds follow from the signs of the coefficients. For x_3, $(\varepsilon_1, \varepsilon_2) = (-1, -1)$ gives 1 and $(1, 1)$ gives 19. For x_4, $(\varepsilon_1, \varepsilon_2) = (1, -1)$ gives the lower bound and $(-1, 1)$ the upper bound. Hence

$$1 \le x_3 \le 19, \tag{7}$$
$$-3 \le x_4 \le 1. \tag{8}$$

ReLU on x_3 (always active). Since $x_3 \ge 1 > 0$, the ReLU can be simplified as:

$$x_5 = \mathrm{ReLU}(x_3) = x_3 = 4\varepsilon_1 + 5\varepsilon_2 + 10, \qquad 1 \le x_5 \le 19.$$

ReLU on x_4 (crossing). Here $l = -3 < 0 < u = 1$, so x_4 crosses zero. For $y = \mathrm{ReLU}(x)$ with $x \in [l, u]$, the bounds are

$$y \ge 0, \qquad y \ge x, \qquad y \le \frac{u}{u - l}\,(x - l).$$

A zonotope can only use two parallel lines. So, with slope $\lambda = \dfrac{u}{u - l}$:

$$\lambda x \;\le\; y \;\le\; \lambda x - \lambda l.$$

Writing $y = \lambda x - \lambda c\, l$ with $c \in [0, 1]$, and $c = \dfrac{\varepsilon_{\text{new}} + 1}{2}$ with $\varepsilon_{\text{new}} \in [-1, 1]$, we get the transformer

$$y = \lambda x - \varepsilon_{\text{new}}\,\frac{\lambda l}{2} - \frac{\lambda l}{2}.$$

For x_4: $l_4 = -3$, $u_4 = 1$, $\lambda_4 = \frac{1}{4}$, so with a fresh noise symbol $\varepsilon_4 \in [-1, 1]$

$$x_6 = \lambda_4 x_4 - \varepsilon_4\,\frac{\lambda_4 l_4}{2} - \frac{\lambda_4 l_4}{2} = \frac{1}{4}(\varepsilon_2 - \varepsilon_1 - 1) + \frac{3}{8}\,\varepsilon_4 + \frac{3}{8}.$$

Proving $x_5 \ge x_6$. Subtracting the two output zonotopes (they share $\varepsilon_1, \varepsilon_2$):

$$\begin{aligned} x_5 - x_6 &= (4\varepsilon_1 + 5\varepsilon_2 + 10) - \left[\tfrac{1}{4}(\varepsilon_2 - \varepsilon_1 - 1) + \tfrac{3}{8}\varepsilon_4 + \tfrac{3}{8}\right] \\ &= \tfrac{17}{4}\varepsilon_1 + \tfrac{19}{4}\varepsilon_2 + \tfrac{41}{4} - \tfrac{3}{8}\varepsilon_4 - \tfrac{3}{8}. \end{aligned}$$

The lower bound is obtained by taking each noise symbol at the end of its range according to their coefficients to minimize the term:

$$x_5 - x_6 \;\ge\; -\tfrac{17}{4} - \tfrac{19}{4} + \tfrac{41}{4} - \tfrac{3}{8} - \tfrac{3}{8} = \tfrac{5}{4} - \tfrac{3}{4} = \tfrac{2}{4} = \tfrac{1}{2}.$$

Therefore

$$x_5 - x_6 \ge \tfrac{1}{2} \ge 0 \quad \Longrightarrow \quad x_5 \ge x_6.$$

So we are able to design the transformers and prove the property.

Problem 3. (MILP encoding for Maxpool)

(a) MILP encoding of $y = \max(x_1, x_2)$

Let $y = \max(x_1, x_2)$ with $x_1 \in [a_1, b_1]$, $x_2 \in [a_2, b_2]$ and $a_1, a_2, b_1, b_2 \in \mathbb{R}$.

For the MILP encoding, we imagine a binary condition and consider two cases.

Case 1: If $x_1 \geq x_2$, then

$$y = x_1,$$
$$y \geq x_2.$$

Case 2: If $x_2 \geq x_1$, then

$$y = x_2,$$
$$y \geq x_1.$$

From both cases, we can see that

$$y \geq x_1 \quad \text{and} \quad y \geq x_2$$

is always true (including the equality). Therefore

$$y \geq x_1, \tag{9}$$
$$y \geq x_2. \tag{10}$$

To upper bound y in terms of of x_1 and x_2, we introduce a binary indicator $\alpha \in \{0, 1\}$, where $\alpha = 1 \implies x_1 \geq x_2$.

Let $u = \max(b_1, b_2)$ denote the maximum of the upper bounds, so that $y \leq u$ always. For an upper bound relation between y and x_1, there needs to be sufficient room above x_1, so that y can lie below the bound. For this, we find the maximum quantity by which y can exceed x_1, which in the case $\alpha = 1 \implies x_1 \geq x_2$ is zero, and in the case $\alpha = 0 \implies x_1 < x_2$ is at most $u - a_1$, since $y \leq u$ and $x_1 \geq a_1$. Hence

$$y \leq x_1 + (1 - \alpha)\,(u - a_1). \tag{11}$$

Similarly, for an upper bound relation between y and x_2, there needs to be sufficient room above x_2. The maximum quantity by which y can exceed x_2 is zero in the case $\alpha = 0$, and in the case $\alpha = 1$ is at most $u - a_2$, since $y \leq u$ and $x_2 \geq a_2$. Hence

$$y \leq x_2 + \alpha\,(u - a_2). \tag{12}$$

Hence, if $\alpha = 1$, constraints (9) and (11) collapse to $x_1 \leq y \leq x_1$, so $y = x_1$, and constraint (10) ensures $y = x_1 \geq x_2$. If $\alpha = 0$, constraints (10) and (12) collapse to $x_2 \leq y \leq x_2$, so $y = x_2$, and constraint (9) ensures $y = x_2 \geq x_1$. In both cases $y = \max(x_1, x_2)$.

(b) Verifying $x_9 > x_{10}$ using the MILP encoding

For a maxpool $y = \max(x_i, x_j)$ with $x_i \in [a_i, b_i]$ and $x_j \in [a_j, b_j]$, the interval bounds of the output are $y \in [\max(a_i, a_j),\ \max(b_i, b_j)]$. For $x_1, x_2 \in [0, 1]$, the interval bounds of the neurons are

$$x_3 = x_1 + x_2 \in [0, 2], \qquad x_4 = x_1 - 2 \in [-2, -1],$$
$$x_5 = x_1 - x_2 \in [-1, 1], \qquad x_6 = x_2 \in [0, 1],$$
$$x_7 \in [0, 2], \qquad x_8 \in [0, 1],$$
$$x_9 = x_7 \in [0, 2], \qquad x_{10} = -x_7 + x_8 - 0.5 \in [-2.5, 0.5].$$

We want to prove that $x_9 - x_{10} > 0$, where

$$x_9 - x_{10} = 2x_7 - x_8 + 0.5.$$

Maxpool $x_7 = \max(x_3, x_4)$**.** The MILP encoding is

$$\begin{aligned} &x_7 \geq x_3, \qquad x_7 \geq x_4, \\ &x_7 \leq x_3 + (1 - \alpha_7)(2 - 0), \\ &x_7 \leq x_4 + \alpha_7\,(2 + 2), \end{aligned}$$

where 2 is the maximum of the upper bounds of x_3 and x_4, and 0 and -2 are the lower bounds of x_3 and x_4, respectively.

The case $\alpha_7 = 0$ is invalid because $x_3 > x_4$: since $x_3 - x_4 = x_2 + 2 \in [2, 3]$, x_3 is always greater than x_4, whereas $\alpha_7 = 0$ would force $x_7 = x_4$, which requires $x_4 \geq x_3$.

In the case $\alpha_7 = 1$, the constraints give $x_7 = x_3$, and the remaining constraints $x_7 \geq x_4$ and $x_7 \leq x_4 + 4$ hold since $x_3 - x_4 \in [2, 3]$.

Maxpool $x_8 = \max(x_5, x_6)$**.** With a binary variable α_8, where $\alpha_8 = 1 \implies x_5 \geq x_6$, the encoding is

$$\begin{aligned} &x_8 \geq x_5, \qquad x_8 \geq x_6, \\ &x_8 \leq x_5 + (1 - \alpha_8)(1 - (-1)), \\ &x_8 \leq x_6 + \alpha_8\,(1 - 0), \end{aligned}$$

where 1 is the maximum of the upper bounds of x_5 and x_6, and -1 and 0 are the lower bounds of x_5 and x_6, respectively.

If $\alpha_8 = 1$, then $x_8 = x_5$ (and $x_8 \leq x_6 + 1$). If $\alpha_8 = 0$, then $x_8 = x_6$ (and $x_8 \leq x_5 + 2$). Since x_5 and x_6 can each be the larger one, both cases must be checked.

Case $\alpha_7 = 1$**,** $\alpha_8 = 1$**.** Here $x_7 = x_3$ and $x_8 = x_5$, so

$$\begin{aligned} x_9 - x_{10} &= 2x_3 - x_5 + 0.5 \\ &= 2(x_1 + x_2) - (x_1 - x_2) + 0.5 \\ &= x_1 + 3x_2 + 0.5 \in [0.5,\ 4.5]. \end{aligned}$$

Hence $x_9 > x_{10}$.

Case $\alpha_7 = 1$**,** $\alpha_8 = 0$**.** Here $x_7 = x_3$ and $x_8 = x_6$, so

$$\begin{aligned} x_9 - x_{10} &= 2x_3 - x_6 + 0.5 \\ &= 2(x_1 + x_2) - x_2 + 0.5 \\ &= 2x_1 + x_2 + 0.5 \in [0.5,\ 3.5]. \end{aligned}$$

Hence $x_9 > x_{10}$.

Since $x_3 \geq x_4$ forces $\alpha_7 = 1$, and $x_9 - x_{10} > 0$ in both cases for α_8, we get $x_9 > x_{10}$ for all $x_1, x_2 \in [0, 1]$. So the MILP analysis can prove the property.

Problem 4. (Programming)

The code is in a Colab notebook **here** and also on **GitHub**.

Network and training. I used the provided network. It was trained on MNIST for 15 epochs using SGD (learning rate 0.001, momentum 0.9) with cross-entropy loss. The training loss went from 2.012 in epoch 1 to 1.540 in epoch 15, and the **clean accuracy on the test set is 93.43%**.

Interval analysis. For an image x with label y and radius ϵ, the input box is $[\max(x - \epsilon, 0),\ \min(x + \epsilon, 1)]$ in every pixel (clamped to the image domain). The box is propagated layer by layer:

- Normalization is monotone (increasing), so it is applied to the lower and upper bounds directly.
- For a linear layer $Wx + b$ with input box of center c and radius r, the output box has center $Wc + b$ and radius $|W|\, r$. (Taken from `https://github.com/Zinoex/bound_propagation`)
- ReLU is monotone, so it is applied to the lower and upper bounds directly.

To verify the property, I add a final linear layer computing $f_y - f_j$ for every $j \neq y$ (row j is $W_y - W_j$ with bias $b_y - b_j$) and compute its lower bound. The image is verified if all nine lower bounds are non-negative, i.e., ≥ 0.

The core of the implementation is the following (the full notebook is linked above):

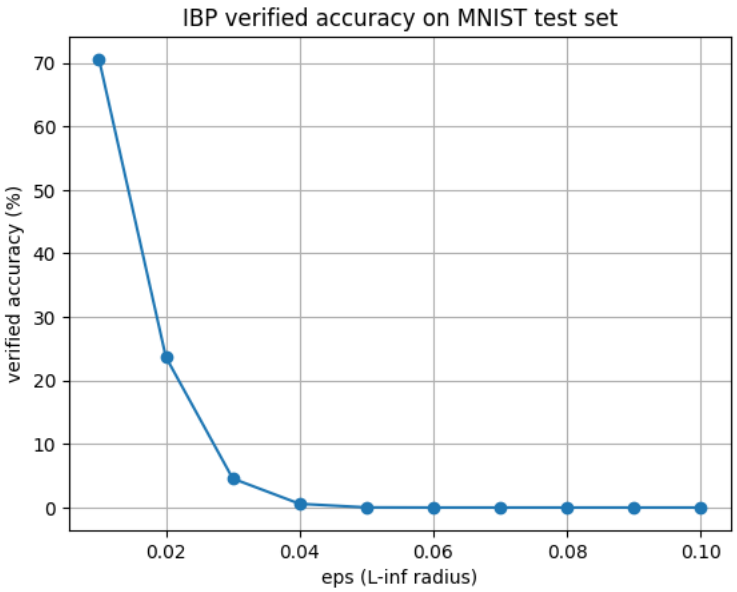
```
def ibp(layer, lower, upper):
    if isinstance(layer, nn.Linear):
        center, rad = (upper + lower) / 2, (upper - lower) / 2
        center = layer(center)                    # W*center + b
        rad = rad @ layer.weight.abs().T          # |W| * rad
        return center - rad, center + rad
    if isinstance(layer, Normalize):
        return layer(lower), layer(upper)         # monotone, so apply to both bounds

def diff_lower_bound(model, x, y, eps):
    norm, net = model[0], model[1]
    lower, upper = (x - eps).clamp(0, 1), (x + eps).clamp(0, 1)
    lower, upper = ibp(norm, lower, upper)
    lower, upper = lower.flatten(1), upper.flatten(1)
    lower, upper = ibp(net.fc, lower, upper)
    lower, upper = F.relu(lower), F.relu(upper)
    # extra output layer: row j is W_y - W_j, bias b_y - b_j
    W, b = net.fc2.weight, net.fc2.bias
    W_diff = W[y].unsqueeze(1) - W.unsqueeze(0)
    b_diff = b[y].unsqueeze(1) - b
    center, rad = (upper + lower) / 2, (upper - lower) / 2
    low = (W_diff * center.unsqueeze(1) - W_diff.abs() * rad.unsqueeze(1)).sum(dim=2) + b_diff
    return low.scatter(1, y.unsqueeze(1), float('inf'))   # ignore j == y
```

An image is verified when the minimum over j of this lower bound is ≥ 0.

Results. Verified accuracy over the whole test set:

ϵ	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09	0.10
Verified acc. (%)	70.55	23.68	4.56	0.58	0.02	0.00	0.00	0.00	0.00	0.00



Verified accuracy drops quickly as ϵ grows: from the clean accuracy of 93.43% to 70.55% at $\epsilon = 0.01$, and it is almost zero from $\epsilon = 0.04$. The average worst-case margin $\min_j(f_y - f_j)$ (lower bound) goes from 1.977 at $\epsilon = 0.01$ to -3.154 at 0.02 and -41.255 at 0.1. Note that a failure to verify does not mean the image is non-robust, it only shows that these bounds were not able to prove it.

```
eps_list = np.linspace(0.01, 0.1, 10)
results = [verify(model, e) for e in eps_list]

print('\n| eps | verified accuracy |\n|---|---|')
for e, r in zip(eps_list, results):
    print(f'| {e:.2f} | {r*100:.2f}% |')

plt.plot(eps_list, [100 * r for r in results], marker='o')
plt.xlabel('eps (L-inf radius)'); plt.ylabel('verified accuracy (%)')
plt.title('IBP verified accuracy on MNIST test set'); plt.grid(True); plt.show()
```

```
eps=0.01 | verified: 70.55% | avg worst-case logit margin: 1.977
eps=0.02 | verified: 23.68% | avg worst-case logit margin: -3.154
eps=0.03 | verified: 4.56% | avg worst-case logit margin: -8.277
eps=0.04 | verified: 0.58% | avg worst-case logit margin: -13.355
eps=0.05 | verified: 0.02% | avg worst-case logit margin: -18.354
eps=0.06 | verified: 0.00% | avg worst-case logit margin: -23.252
eps=0.07 | verified: 0.00% | avg worst-case logit margin: -28.018
eps=0.08 | verified: 0.00% | avg worst-case logit margin: -32.621
eps=0.09 | verified: 0.00% | avg worst-case logit margin: -37.033
eps=0.10 | verified: 0.00% | avg worst-case logit margin: -41.255
```

Problem 5. (Certifier Design in ConstraintFlow)

(a) A new certifier

The new certifier (`mycert.cf`) extends DeepPoly by keeping two polyhedral lower bounds for each neuron. The shape has an extra polyhedral lower bound `La` next to `l`, `u`, `L` and `U`:

```
def Shape as (Float l, Float u, PolyExp L, PolyExp La, PolyExp U){[(curr[l]<=curr),(curr[u]>=curr
    ),(curr[L]<=curr),(curr[La]<=curr),(curr[U]>=curr)]};
```

If the input of a ReLU crosses zero, both 0 and x are sound lower bounds. DeepPoly selects one of them with the area heuristic (`relu_lower`, stored in `L`), and we store the opposite choice in `La` (`relu_lower2`). The Affine transformer back-substitutes twice, and the two chains differ only in which polyhedral lower bound they use, `L` or `La`. It keeps the tighter result: the maximum of the two lower bounds for l and the minimum of the two upper bounds for u.

```
func relu_lower(Neuron n)  = n[l] + n[u] >= 0.0 ? n : 0.0;
// The opposite choice of the lower bound is stored in La.
func relu_lower2(Neuron n) = n[l] + n[u] >= 0.0 ? 0.0 : n;

transformer deeppoly{
    Affine -> (max(backsubs_lower1(prev.dot(curr[weight]) + curr[bias], curr),
                   backsubs_lower2(prev.dot(curr[weight]) + curr[bias], curr)),
               min(backsubs_upper1(prev.dot(curr[weight]) + curr[bias], curr),
                   backsubs_upper2(prev.dot(curr[weight]) + curr[bias], curr)),
               prev.dot(curr[weight]) + curr[bias],
               prev.dot(curr[weight]) + curr[bias],
               prev.dot(curr[weight]) + curr[bias]);
    Relu -> (relu(prev[l]), relu(prev[u]), relu_lower(prev), relu_lower2(prev), relu_upper(prev))
    ;
}
```

The novelty is the second lower bound `La`, which gives us a second back-substitution chain. The first chain is exactly DeepPoly's, so our certifier is at least as tight as DeepPoly. The second chain uses the other choice of lower bound for the ReLU, so it can perhaps give a tighter bound than the first one. Both chains are sound, so taking the maximum of the two lower bounds and the minimum of the two upper bounds is also sound, and it is tighter. The maximum of two sound lower bounds is at least as large as each of them, and the minimum of two sound upper bounds is at most as large as each of them. The full file is at `mycert.cf`.

(b) Soundness check

We ran `constraintflow provesound .\mycert.cf`. The output is:

```
(cfenv) PS H:\Atharva Bhatt\UIUC MSCS Fall 2026\CS 521 Trustworthy AI Systems\CCS521FA26HW\hw2\q5> constraintflow provesound
.\mycert.cf
H:\Atharva Bhatt\UIUC MSCS Fall 2026\CS 521 Trustworthy AI Systems\CCS521FA26HW\hw2\q5\cfenv\Lib\site-packages\z3\z3core.py:5
: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_re
sources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  import pkg_resources
Proved Affine
Proved Relu
Provesound result: None
```

(c) Results on MNIST

We ran our certifier on the first 100 MNIST test images for $\epsilon \in \{0.005, 0.01, 0.02, 0.03, 0.05\}$ with the following command:

```
CF_NO_PARALLEL=1 python run_cf.py mycert.cf --num-images 100 \
    --eps 0.005 0.01 0.02 0.03 0.05 --init La=L la=l ua=u
```

The compiler only initializes the shape members `l`, `u`, `L`, `U` and `Z` at the input layer, so `--init` is needed to initialize the new member `La` as a copy of `L`. `CF_NO_PARALLEL=1` is needed because the parallelization pass assumes exactly two traverse loops in a transformer and ***errors out*** if there are more (see the comment in `mycert.cf`). The percentage of verified images and the time per image are:

	DeepPoly		Our certifier	
ϵ	Verified (%)	Time/img (s)	Verified (%)	Time/img (s)
0.005	99	0.4392	99	0.8176
0.01	94	0.4383	94	0.8205
0.02	83	0.4207	83	0.8333
0.03	49	0.4241	49	0.8350
0.05	2	0.4224	2	0.8318

The output of original DeepPoly certifier is:

```
>> python run_cf.py deeppoly.cf --num-images 100 --eps 0.005 0.01 0.02 0.03 0.05
H:\Atharva Bhatt\UIUC MSCS Fall 2026\CS 521 Trustworthy AI Systems\CCS521FA26HW\hw2\q5\cfenv\Lib\site-packages\z3\z3core.py:5: UserWarni
ng: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is sla
ted for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  import pkg_resources
Network: mnist_relu_3_100.onnx
Clean accuracy on 100 images: 99.00%
     eps   verified   time/img (s)
   0.005     99.00%        0.4392
    0.01     94.00%        0.4383
    0.02     83.00%        0.4207
    0.03     49.00%        0.4241
    0.05      2.00%        0.4224
```

The output of our certifier is:

```
(cfenv) PS H:\Atharva Bhatt\UIUC MSCS Fall 2026\CS 521 Trustworthy AI Systems\CCS521FA26HW\hw2\q5> $env:CF_NO_PARALLEL = "1"
>> python run_cf.py mycert.cf --num-images 100 --eps 0.005 0.01 0.02 0.03 0.05 --init La=L la=l ua=u
H:\Atharva Bhatt\UIUC MSCS Fall 2026\CS 521 Trustworthy AI Systems\CCS521FA26HW\hw2\q5\cfenv\Lib\site-packages\z3\z3core.py:5: UserWarni
ng: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is sla
ted for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  import pkg_resources
Network: mnist_relu_3_100.onnx
Clean accuracy on 100 images: 99.00%
     eps   verified   time/img (s)
   0.005     99.00%        0.8176
    0.01     94.00%        0.8205
    0.02     83.00%        0.8333
    0.03     49.00%        0.8350
    0.05      2.00%        0.8318
(cfenv) PS H:\Atharva Bhatt\UIUC MSCS Fall 2026\CS 521 Trustworthy AI Systems\CCS521FA26HW\hw2\q5>
```

The results are the same as DeepPoly's. Our certifier is at least as tight as DeepPoly, but an image is verified only if the bound becomes precise enough to prove the true label, so a tighter bound changes the result only when it crosses that threshold. The first chain of our certifier is exactly DeepPoly's. The second chain uses the opposite lower bound at every ReLU that crosses zero, whereas DeepPoly's heuristic already chooses the lower bound with the smaller area for each ReLU. So, the second chain can be looser than the first, and then choosing a tighter bound falls back to DeepPoly's bound. On these 100 images, any improvement from the second chain was not enough to verify any additional image. Our certifier takes about twice as long per image (about 0.82 s versus 0.42 s), since the Affine transformer back-substitutes twice.

The mycert.cf certifier file is available **here**.

(d) HardTanh transformer

The function is

$$\text{HardTanh}(x) = \begin{cases} -1 & \text{if } x \leq -1, \\ x & \text{if } -1 < x < 1, \\ 1 & \text{if } x \geq 1. \end{cases}$$

HardTanh is monotone, so the concrete bounds of the output are obtained by applying HardTanh to the concrete bounds l_x and u_x of the input: $l_y = \text{HardTanh}(l_x)$ and $u_y = \text{HardTanh}(u_x)$. The polyhedral bounds depend on where the input interval $[l_x, u_x]$ lies. The lower bound is

$$L_y = \begin{cases} 1 & \text{if } l_x \geq 1, \\ x & \text{if } l_x \geq -1 \text{ and } u_x \leq 1, \\ -1 & \text{otherwise,} \end{cases}$$

and the upper bound is

$$U_y = \begin{cases} -1 & \text{if } u_x \leq -1, \\ x & \text{if } l_x \geq -1 \text{ and } u_x \leq 1, \\ 1 & \text{otherwise.} \end{cases}$$

The Affine transformer is kept from `deeppoly.cf`.

```
func hardTanh(Float x) = max(-(1.0), min(1.0,x));
func hardTanh_lower(Neuron n) = n[l] >= 1.0 ? 1.0 : (((n[u] <= 1.0) and (n[l] >= -(1.0))) ? n :
    -(1.0));
func hardTanh_upper(Neuron n) = n[u] <= -(1.0) ? -(1.0) : (((n[l] >= -(1.0)) and (n[u] <= 1.0)) ?
     n : 1.0);

HardTanh -> (hardTanh(prev[l]), hardTanh(prev[u]), hardTanh_lower(prev), hardTanh_upper(prev));
```

We ran `constraintflow provesound .\deeppoly.cf`. The output is:

```
(cfenv) PS H:\Atharva Bhatt\UIUC MSCS Fall 2026\CS 521 Trustworthy AI Systems\CCS521FA26HW\hw2\q5> constraintflow provesound
.\deeppoly.cf
H:\Atharva Bhatt\UIUC MSCS Fall 2026\CS 521 Trustworthy AI Systems\CCS521FA26HW\hw2\q5\cfenv\Lib\site-packages\z3\z3core.py:5
: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_re
sources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  import pkg_resources
Proved Affine
Proved Relu
Proved HardTanh
Provesound result: None
```

The HardTanh certifier file is available **here**.